

Skill 3 · Base de Conversão LSP → Java

Versão: v1.5 · Interna · `skill-03-base-conversao-lsp-java.md`

Skill interna — **não** é fluxo de usuário. Aplique as regras globais do Router. Em conflito de assinatura, **revalide na Skill 2 / página oficial**.

Fronteira: Skill 2 = links/alias; **esta skill** = mecânica + catálogo + padrões de compilação + exemplos.

Precedência: docs oficiais Skill 2 → catálogo oficial abaixo → padrões `padrao_compilacao` (Eclipse) → anexos sanitizados → `validacao_manual` .

Crescimento do catálogo confirmada : só com doc oficial / Skill 2. Achados de compilação entram como `padrao_compilacao` , não como inventados.

Como consultar (ordem obrigatória)

- A. Restrições + Regra de ouro + Invariantes ← nunca pular
- B. Workflow (6 passos)
- C. Tipos / sintaxe / VaPara / Funcao (se necessário)
- D. Catálogo – só a família do item LSP
- E. Acesso a dados (USU_* / R* / ContextSession) se houver cursor/SQL
- F. Armadilhas Eclipse + checklist
- G. Exemplos – só o padrão análogo
- H. Saída tipada para a Skill 1

Não leia o catálogo inteiro de ponta a ponta. Localize a **família** no índice. Em conflito de assinatura, **revalide na Skill 2**.

Índice rápido do arquivo

Precisa de...	Vá para
Proibições / minutos / Sem Situacao.getMinutos	Restrições absolutas
Modelo mental LSP→contexto	Regra de ouro
Passo a passo	Workflow
Tipagem / Se→if / VaPara / Funcao	Tipos e sintaxe
Esqueleto Apuracao / Consistência / BH	Esqueletos + tipos de regra
DatPro, HorSit, FPxMar, históricos...	Catálogo → família
TipCon / MarcacaoRegra / overloads HorSit	Catálogo + notas <code>padrao_compilacao</code>
Cursor USU_* / .sc / ContextSession	Acesso a dados
Erros Eclipse / anti-padrões	Armadilhas
CalculaQtdMinutos / interjornada / TipCon SQL	Exemplos sanitizados
Templates IEntity / .sc / DBCenter	Acesso a dados

VaPara / operadores	Tipos e sintaxe
InicioCalculo / AposGravar	Outros pontos
⚠ pendente	Itens pendentes
Campos LSP conflitantes	Tabela conflitantes

Fontes oficiais

Fonte	URL	Uso
Equivalência das funções de regras	https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/informacoes-adicionais/rotinas/gpo/integracao-controle-ponto-refeitorio/equivalencia-funcoes-regras.htm	Mapa oficial
Índice das Funções HCM 6.10.4	https://documentacao.senior.com.br/gestao-de-pessoas-hcm/6.10.4/customizacoes/funcoes.htm	Assinatura + contexto do método

Catálogo abaixo: evidência `confirmada` (salvo nota). Exemplos: `padrao_anexo`.

Assinatura/contexto incompletos no catálogo → abrir Índice das Funções (Skill 2) antes de gerar código.

Quando usar / não usar

Usar	Não usar
Converter/mapear variáveis e funções de apuração LSP→Java	Só mentoria; inventar método; só citar link (Skill 2)

Restrições absolutas

1. Preferir o catálogo; se faltar → Skill 2 → `validacao_manual` (não inventar).
2. Horas em APIs de ponto = **minutos inteiros** (`int` , não `double` solto).
3. SQL/cursor → API semântica do catálogo **antes** de `EntitySession/ContextSession` — exceto tabela **USU_* custom** sem API (aí `ICursor+ .sc`).
4. **Proibido** `getSituacao(...).getMinutos()` / `setMinutos(...)` — use `getHorSit` / `setHorSit` / `zeraHorasSituacao` .
5. Sanitize nomes de cliente em exemplos anexados; package do projeto é **parâmetro** (ex. sanitizado `gestaopontoCustom`) — nunca vaziar cliente.
6. Sem `Mensagem()` / `popup` em apuração — preferir `mensagemLog` .
7. Confirmar se o método existe no **contexto** da regra (apuração ≠ consistência ≠ BH ≠ geral).
8. `execute()` **sem** `throws Exception` ; auxiliares `private` no **nível da classe** (nunca aninhados).
9. `ContextSession/DBCenter` da plataforma ≠ Senior SQL 2 (SQL 2 continua proibido).

Regra de ouro

LSP e Java diferem no **modelo de execução**: variáveis/funções globais LSP → **métodos no objeto de contexto**.

Quem só troca `Inicio/Fim` por `{ }` produz código que não compila.

Workflow (6 passos)

1. Identificar contexto (apuração / consistência / bloqueio / fechamento BH / geral).
2. Inventariar construções LSP.
3. Mapear no catálogo (nome e ordem de parâmetros).
4. Traduzir mecânica (getters/setters, arrays→métodos, cursor→ `List` , `End` →retorno).
5. Traduzir sintaxe (`Se` → `if` , `=` comparação→ `==` / `.isEqual()` , etc.).
6. Revisar armadilhas.

Invariantes

1. Toda variável/função de contexto LSP → chamada de método.
2. Não inventar assinatura.
3. Conferir ordem de parâmetros.
4. Horas = minutos inteiros.
5. Tipagem forte Java.
6. Sem popup em apuração.
7. Arrays indexados → métodos/coleções (não `getApuDiu(1)` inventado).
8. Método deve existir no **contexto** da regra.

Tipos e sintaxe

LSP	Java
Numero inteiro	<code>int</code> / <code>long</code>
Numero decimal	<code>double</code>
Alfa	<code>String</code>
lógico 0/1	<code>boolean</code>
Data	<code>LocalDate</code> (ou padrão do SDK — não misturar Joda + <code>java.time</code> sem necessidade)
Hora (minutos)	<code>int</code> minutos
marcação	<code>MarcacaoRegra</code> (<code>padrao_compilacao</code> ; preferir a <code>Marcacao</code> genérica)
<code>Se</code> / <code>Senao</code> / <code>Enquanto</code> / <code>Inicio/Fim</code>	<code>if</code> / <code>else</code> / <code>while</code> / <code>{ }</code>
Vapara / labels	<code>early return</code> ; <code>break/continue</code> ; ou <code>flag + do { }</code> <code>while(false)</code>
Funcao sem end	<code>private void</code> no nível da classe
Funcao com 1 end	retorno tipado do método
Funcao com vários end	<code>int[]</code> / objeto (não closures aninhadas)
Numero[n] array	<code>int[n+1]</code> ou <code>double[n+1]</code> (LSP 1-based → Java 0-based)
Tabela	<code>List<record></code> / classe interna
@ ... @	<code>//</code>

= comparação	== / .isEqual() / .equals()
<>	!=
decimal ,	.
e / ou / nao	&& / `
RestoDivisao(a,b)	a % b
Mensagem(...) em regra	exceção de domínio (BusinessException / RegraApuracaoException) — não popup

VaPara — 3 padrões (padrao_compilacao)

1. Early return

```
Se (condicao) { VaPara FimRegra; }
@ lógica
FimRegra:
```

```
if (condicao) return;
// lógica
```

2. Flag + do-while false (bloco “pulado” com label)

```
boolean fezProrrogacao = false;
do {
    // bloco principal
    fezProrrogacao = true;
} while (false);
if (!fezProrrogacao) { /* bloco do label */ }
```

3. Break de loop

```
while (condicao) {
    if (encerrar) break;
}
```

Para (0-based em Java)

```
for (int i = 0; i < n; i++) { /* LSP Para costuma 1..n */ }
```

Instruções

1. Seguir “Como consultar” (A→H)
2. Identificar contexto
3. Buscar item só na família do Catálogo

4. Assinatura/contexto duvidosos → Índice das Funções (Skill 2)
5. Ausente → validacao_manual

Tipos de regra e esqueletos (padrao_compilacao)

Tipo LSP	Classe base	Contexto	
Apuração	custom.senior.apuracao.Apuracao	getContextoApuracao()	Regra
Consistência	custom.senior.apuracao.ConsistenciaAcertos	getContextoConsistenciaAcerto()	Regra
Manutenção BH	custom.senior.bancohoras.RegraManutencaoBH	getContextoManutencaoBH()	Regra
Fechamento BH	custom.senior.bancohoras.RegraFechamentoBH	getContextoFechamentoBH()	Regra

Package: o informado pelo usuário/projeto. Exemplo sanitizado: `gestaopontoCustom` (não `gestaoDoPontoCustom`).

Apuração

```
@Rule(description = "Regra de Apuracao")
public class RegraApuracao extends Apuracao {
    @Override
    public void execute() { // SEM throws Exception
        ContextoGeralRH ctxGeral = getContainer().getContextoGeral();
        ContextoApuracao ctx = getContainer().getContextoApuracao();
    }

    private void auxiliar(ContextoApuracao ctx, int param) { /* nível da classe */ }
}
```

Consistência de Acertos

```
@Rule(description = "Consistencia de Acertos")
public class RegraConsistencia extends ConsistenciaAcertos {
    @Override
    public void execute() {
        ContextoConsistenciaAcerto ctx =
getContainer().getContextoConsistenciaAcerto();
        // extras: getHorSitAnterior, getDataInicial/Final, getHorSitAnteriorFaixa
    }
}
```

Fechamento BH

```
@Rule(description = "Fechamento BH")
public class RegraFechamentoBH extends custom.senior.bancohoras.RegraFechamentoBH {
```

```

@Override
public void execute() {
    ContextoFechamentoBH ctx = getContainer().getContextoFechamentoBH();
}
}

```

Imports frequentes (padrao_compilacao): java.time.LocalDate / LocalDateTime , com.senior.rule.Rule , com.senior.dataset.ICursor , MappedParamProvider , Colaborador , MarcacaoRegra , TipoIntervalo , TipoHoraExtra , ContextSession , DBCenter , IResultSet , com.senior.dataset.IEntity (+ @Entity / @Field), readonly IR030EMP / IR060DSI /...

Catálogo de equivalência (oficial Senior 6.10.4)

Fonte: [Equivalência das funções de regras](#).

Métodos tipicamente em contextoApuracao / container — **confirmar contexto e assinatura** no Índice das Funções.

Famílias (âncora de busca)

Família	Itens típicos LSP
Data / sistema	DatPro, HorSis, DiaSem, DatIni...
Situações	HorSit[], SitAnt[], TotSit[], MotSit
Horas apuradas/previstas	ApuDiu[], ApuNot[], TraDiu, PrvTra[]...
Marcações	FPxMar, FLeMar, DatMar, QtdMar...
Escala / horário / feriado	EscAtu, CodHor, FerFil...
Históricos / usuário	CarEmp, CodSinEmp, CodUsu...
Compensação / BH / log	TemCmp[], RetBHRDat, MensagemLog...

Data / sistema

LSP (Controle de Ponto)	Java (Gestão do Ponto)
DatPro (implícito via Ano/Dia/MesPro)	getData()
AnoPro	getAnoData(Date data) / getData()
MesPro	getMesData(Date data)
DiaPro	getDiaData(Date data)
AnoSis / MesSis / DiaSis	getAnoData / getMesData / getDiaData
HorSis / ExtSis	Nativo em Java
DatIni / Datfim	getDataInicial() / getDataFinal()

DiaSem, DiaDom...DiaSab	Preferir <code>getDiaSem(Date)</code> ; alt. <code>getData().getDayOfWeek().getValue()</code> (<code>padrao_compilacao</code>)
MesAtu	<code>getMesData(Date data)</code>
RetornaAnoData / RetornaDiaData / RetornaMesData	<code>getAnoData / getDiaData / getMesData</code>

Situações / HorSit

LSP	Java
HorSit[]	<code>getHorSit(int) / getHorSit(LocalDate, int), setHorSit, somaHorasSituacao, zeraHorasSituacao, zeraHorasSituacaoFaixa, getHorSitFaixa</code> — overloads <code>padrao_compilacao: getHorSit(int[]), zeraHorasSituacao(varargs)</code>
SitAnt[]	<code>getHorSitAnterior(int) / getHorSitAnteriorFaixa</code> (consistência)
TotSit[] / BuscaTotalizadoresSituacoes	<code>getTotalSituacoes(int codigoTotalizador, date data)</code> (e overload com intervalo)
MotSit	<code>getMotivoAcerto(int situacao)</code>

Apuração diurna/noturna / trabalhadas / previstas

LSP	Java
ApuDiu[] / ApuNot[]	<code>getHorasSeparadas(TipoIntervalo TipoHoraExtra) → ISeparacaoHoras (getHorasDiurnas/Noturnas/TotalHoras);</code> OU <code>getHoras(MarcacaoRegra, ...)</code>
HrtRaD[] / HrtRan[] / TraDiu / TraNot	<code>getHorasTrabalhadas(int parte)</code>
MinPvd[] / MinPvn[] / PrvTra[] / PrvTrd / PrvTrn / VprVho / VprVin / VprVtr	<code>getTotalMinutosPrevisto(...)</code> / <code>getHorasPrevistas(...)</code> / <code>getTotalMinutosPrevistoProrrogado(...)</code>
FatRed	<code>getHorarioPrevistoColaborador(...)</code>
ExtrasIntervalo	<code>getExtrasIntervalo / getExtrasIntervaloAnterior / getExtrasIntervaloPosterior</code>
MinExD[] / MinExn[] / MsaInd[] / MsaInn[] / IniExt / FimExt / PerExt / PriExt / QtdExt / TipExt / DiaExt	<code>getIntervalosCalculados()</code> / <code>getIntervaloCalculado(int indice)</code>

Marcações

LSP	Java
FPxMar / FLeMar / ...	<code>List<MarcacaoRegra> getMarcacoesRealizadas(boolean)</code> — <code>getData()</code> → <code>.toLocalDate()</code> ; índice Java 0-based

QtdMar / TotMar	Preferir getQtdMarcacoesRealizadas(boolean); alt. .size() (padrao_compilacao)
DulMar / HulMar	getMarcacaoAnterior() → tipo MarcacaoAnterior: só diferencaMinutos(data, hora) — sem getHora/getData
MinJor	Preferir getHorasInterjornadaRealizada(); alt. diferencaMinutos se só MarcacaoAnterior
MinMJo	getHorasInterjornadaPrevista()
TipCon	Não existe Colaborador.getTipCon() — SQL SELECT TIPCON FROM R034FUN ... via ContextSession (padrao_compilacao)
ApuDiu/ApuNot tipados	Também getHorasSeparadas(TipoIntervalo TipoHoraExtra) → ISeparacaoHoras
log / pendência / alerta	mensagemLog, setGerarPendencia, criarAlerta (try/catch)

Escala / horário / filial / feriado

LSP	Java
EscAtu	getEscala()
EscEmp / RetEscEmp	getHistoricoEscala()
EscTrf	getEscalaHistorico()
TemTes	getTrocaEscala(Date data)
TemThr	getTrocaHorario(Date data)
RetornaEscala	getEscalaPrevistaColaborador(...)
CodHor / RetornaHorarioApurado	getHorario() / getHorario().getCodigo() — especiais 9996–9999 (folga/feriado/comp/DSR)
HorEsc / HorTrf	getHorarioEscala()
HorDFe	getHorarioOriginalEscala()
HorFol	getCodigoHorarioFolga() / getHorarioFolga()
HorPfo	getHorarioProjecaoFolga()
RetornaHorario / RetornaBatidaHorario	getHorarioPrevistoColaborador(...)
NumPer	getNumeroPeriodos(int codHor)
NumInt	getNumeroIntervalos()
NinRef	getNumeroIntervaloRefeicao()
TurInt	getTurmaIntervalo()

RetMinRefHTr	getMinutosRefeicaoPrevisto()
FilEmp / RetFilEmp / DatAltFil / EmpAltFil	getHistoricoFilial()
FerFil	getFeriadoFilial(Date data)
VerDatFer	getFeriado(Date data)

Históricos colaborador / usuário

LSP	Java
CarEmp / EstCarEmp / RetCarEmp	getHistoricoCargo()
CcuEmp / DatAltCcu / RetCcuEmp	getHistoricoCentrodeCusto()
CodSinEmp / RetSinEmp	getHistoricoSindicato()
CodVinEmp / RetVinEmp	getHistoricoVinculo()
LocEmp / RetLocEmp	getHistoricoLocal(...)
CodAfs / IniAfs / FimAfs / TemAfs / QtdAfs[]	getHistoricosAfastamento()
BusCraTit	getHistoricosCracha() / getHistoricosCrachaProvisorio()
RetApuPon	getHistoricoApuracao(colaborador, data)
CodUsu	getUsuarioAtivo()
NomUsu	getUsuario(long codigoUsuario)
RetornaDesGrupo / RetornaQtdGrupos	getGrupos(long codigoUsuario)
AssociaUsuColab	associarUsuarioColaborador(...)
RetColabPorCodUsu	getUsuarioColaborador(...)

Compensação / banco de horas / cálculo

LSP	Java
TemCmp[] / DtICmp[] / DtFCmp[] / PerCmp[] / QtdCmp[] / SitCmD[] / SitCmN[] / TipCmp[]	getCompensacoes() / getCompensacoes(LocalDate data)
LimBa1 / LimBa2	getEscala()
RetBHRDat	getSaldoBanco(int banco, int empresa, int tipo, int cadastro, LocalDate data)
TipCal	getTipoCalculo()
MensagemLog	mensagemLog(String mensagem)
VerificaAbrangenciaNumero	getAbrangencia(string abrangencia, int numero)

RetornaCodLoc / RetornaNumLoc / RetNivLoc	getCodigoLocal / getNumeroLocal / getQtdNiveisLocal
---	---

Definição de situações (padrão complementar)

Cursor R014SIN / R030EMP para CodDsi → preferir API (padrao_anexo até confirmar na doc):

```
int codDsi = contextoApuracao.getDefinicaoSituacoes().getCodigo();
```

Campos LSP conflitantes (padrao_compilacao)

LSP	Erro comum	Correto
TipCon	col.getTipCon()	SQL R034FUN.TIPCON
nMar / marcações	índice 1-based / tipo errado	List<MarcacaoRegra> 0-based + .toLocaleDate()
MarcacaoAnterior	getHora() / getData()	só diferencaMinutos
HorSit[n]	tratar como array Java	get/setHorSit(codSit, minutos)
SitAnt	getter inventado	getHorSitAnterior no contexto de consistência
QtdMar	inventar campo	API oficial ou .size()

Domínio HCM — constantes úteis

Valor	Uso
CodHor 9996–9999	Folga / feriado / compensado / DSR (<9996 = horário normal)
1320 / 300	Janela noturna 22h–05h (minutos)
660 / 360	Limites CLT interjornada / sem intervalo
1440	Virada de dia em CalculaQtdMinutos

Acesso a dados (após API-first)

```
Tabela USU_* custom?  
SIM → IEntity + .sc + ICursor (EntitySession)  
NÃO (R* nativa / USU_* em tabela nativa) → ContextSession SELECT ou DBCenter DML  
Readonly em com.senior.rh.entities.readonly.*? → importar, não recriar
```

Fatal: .sc apontando para Rxxxxx → "Tabela R034FUN inválida. É permitido mapear somente tabelas de usuário".

ICursor — uma linha

```

MappedParamProvider params = new MappedParamProvider();
params.setParam("vNumEmp", vNumEmp);
ICursor<IMinhaTabela> cur =
getContainer().getEntitySession().newCursor(IMinhaTabela.class);
cur.addFilter("USU_NumEmp = :vNumEmp", params);
cur.open();
try {
    if (cur.first()) { IMinhaTabela row = cur.read(); }
} finally { cur.close(); }

```

ICursor — múltiplas linhas + ordem

```

String[] campos = new String[]{"USU_DatAlt"};
OrderDirection[] ordem = new OrderDirection[]{OrderDirection.DESC};
cur.setOrder(campos, ordem);
cur.open();
try {
    while (cur.next()) {
        IMinhaTabela row = cur.read();
    }
} finally { cur.close(); }

```

Três formas de EntitySession

```

// 1) preferencial no execute()
getContainer().getEntitySession().newCursor(T.class);

// 2) auxiliar fora do execute()
IEntitySession es = EntitySessionProvider.getSession();
es.newCursor(T.class);

// 3) chave exata (CursorUtil)
ICursor<T> cur = CursorUtil.getCursor(T.class);
T buffer = cur.newBuffer();
buffer.setNumEmp(col.getNumEmp());
cur.open();
try {
    if (cur.search(buffer, SearchMode.EXACT_MATCH)) { cur.read(buffer); }
} finally { cur.close(); }

```

ContextSession (SELECT) vs DBCenter (DML)

	ContextSession	DBCenter
Uso	SELECT	INSERT/UPDATE/DELETE
Close	Não fechar (container)	Fechar no finally
Placeholder	?	?

Índice IResultSet	base-0 (getInt(0) = 1º campo)	—
databaseId	—	ex. "vetorh" (confirmar projeto)

DBCenter INSERT (sanitizado)

```

IDBCenter database = DBCenter.getInstance("vetorh"); // confirmar databaseId do
projeto
ISession session = null;
try {
    session = database.newSession();
    Date dataConv =
Date.from(dataPro.atStartOfDay(ZoneId.systemDefault()).toInstant());
    String dataStr = new SimpleDateFormat("dd/MM/yyyy").format(dataConv);
    session.executeUpdate(
        "INSERT INTO USU_TExemplo (USU_NumEmp, USU_Dat, USU_CodSit) VALUES (?, ?,
?),",
        numEmp, dataStr, codSit);
} catch (Exception ex) {
    ctx.mensagemLog("Erro DML: " + ex.getMessage());
} finally {
    if (session != null) session.close();
}

```

Template IEntity (só USU_*)

```

package gestaopontoCustom; // package do projeto – sanitizar

import java.time.LocalDate;
import com.senior.dataset.IEntity;
import com.senior.dataset.annotation.Entity;
import com.senior.dataset.annotation.Field;

@Entity
public interface IMinhaTabela extends IEntity {

    @Field(description = "Empresa")
    int getUSU_CodEmp();
    void setUSU_CodEmp(int USU_CodEmp);
    boolean isUSU_CodEmpNull();
    void setUSU_CodEmpNull();

    @Field(description = "Data")
    LocalDate getUSU_DatEmi();
    void setUSU_DatEmi(LocalDate USU_DatEmi);
    boolean isUSU_DatEmiNull();
    void setUSU_DatEmiNull();

    @Field(description = "Usuário – long")
    long getUSU_CodUsu();
}

```

```

    void setUSU_CodUsu(long USU_CodUsu);
    boolean isUSU_CodUsuNull();
    void setUSU_CodUsuNull();
}

```

Null: `int v = entity.isUSU_CodEmpNull() ? 0 : entity.getUSU_CodEmp();`

IDs de usuário/consultor → **long**, não **int**.

Import correto: `com.senior.dataset.IEntity` — **nunca** `com.senior.g5...IEntity`.

Template .sc (JSON puro)

```

{
  "datasets": [
    {
      "id": "ScMinhaTabela",
      "entity": "gestaopontoCustom.IMinhaTabela",
      "storage": {
        "databaseId": "vetorh",
        "query": "from USU_MINHATAB USU_MINHATAB",
        "fieldMapping": [
          { "field": "USU_CodEmp", "column": "USU_MINHATAB.USU_CodEmp" },
          { "field": "USU_DatEmi", "column": "USU_MINHATAB.USU_DatEmi" },
          { "field": "USU_CodUsu", "column": "USU_MINHATAB.USU_CodUsu" }
        ],
        "type": "DB"
      }
    }
  ]
}

```

Regras: `id` = nome do arquivo sem extensão; começa com `{`; sem BOM/comentários; só `USU_*` no `query`.

Readonly do framework (importar)

`IR030EMP`, `IR060DSI`, `IR004HOR`, `IR010SIT`, `IR010TOB`, `IR014SIN`, `IR070ACC`, ...

Getters encadeados / ISeparacaoHoras (padrao_compilacao)

```

int codSin = ctx.getHistoricoSindicato().getCodSin();
int tabOrg = ctx.getHistoricoLocal().getTabOrg();
int numLoc = ctx.getHistoricoLocal().getNumLoc();
HistoricoVinculo hv = ctx.getHistoricoVinculo();
if (hv != null) { /* usar hv */ }

ISeparacaoHoras sep = ctx.getHorasSeparadas(TipoIntervalo.REFEICA0);
int diu = sep.getHorasDiurnas();
int not = sep.getHorasNoturnas();
int tot = sep.getTotalHoras();

```

Armadilhas práticas

Armadilha / erro Eclipse	Correção
Só trocar sintaxe sem mapear variáveis	Inventário + getters/setters primeiro
Copiar ordem de parâmetros LSP	Confirmar no Índice das Funções
<code>getSituacao().get/setMinutos</code>	<code>getHorSit / setHorSit / zeraHorasSituacao</code>
Cursor SQL por reflexo	Buscar família no catálogo; <code>USU_*</code> → <code>ICursor</code>
<code>getHorSit(variavelDeMinutos)</code>	1º arg = código da situação
Inventar <code>getApuDiu(1)</code>	<code>getHoras / getHorasSeparadas</code>
<code>execute()</code> throws <code>Exception</code>	Remover throws (<code>IRulePoint</code>)
Método aninhado / código solto	Auxiliares no nível da classe
<code>col.getTipCon()</code>	<code>SQL R034FUN.TIPCON</code>
<code>MarcacaoAnterior.getHora/getData</code>	<code>diferencaMinutos</code>
<code>LocalDateTime</code> → <code>LocalDate</code>	<code>.toLocalDate()</code> em <code>MarcacaoRegra.getData()</code>
<code>.sc</code> em tabela <code>R*</code>	Remover <code>.sc</code> ; <code>ContextSession</code>
<code>.sc</code> não-JSON / BOM	Reescrever começando com <code>{</code>
<code>import com.senior.g5...IEntity</code>	<code>com.senior.dataset.IEntity</code>
Package <code>gestaoDoPontoCustom</code>	Package do projeto (ex. <code>gestaopontoCustom</code>)
<code>IRResultSet</code> base-1	base-0
Filename ≠ classe pública	Renomear arquivo (Windows: temp rename)
Inventar método	<code>validacao_manual</code>
Misturar <code>java.time</code> e <code>Joda</code>	Preferir <code>java.time</code> / SDK

Mensagens Eclipse literais (referência)

Mensagem	Correção
The import <code>com.senior.g5</code> cannot be resolved	<code>import com.senior.dataset.IEntity</code>
<code>execute()</code> throws <code>Exception</code> not declared by <code>IRulePoint</code>	Remover throws <code>Exception</code>
The method <code>getTipCon()</code> is undefined for the type <code>Colaborador</code>	<code>SQL R034FUN.TIPCON</code>
The method <code>getHora()</code> is undefined for the type <code>MarcacaoAnterior</code>	<code>diferencaMinutos</code>
Type mismatch: cannot convert <code>LocalDateTime</code> to <code>LocalDate</code>	<code>.toLocalDate()</code>

Expected BEGIN_OBJECT but was STRING	.sc JSON puro começando com {
Tabela Rxxx inválida. É permitido mapear somente tabelas de usuário	Sem .sc em R*
The public type X must be defined in its own file	Filename = nome da classe

Windows (case): Refactor → nome temporário → nome correto (ou `ren` em duas etapas no CMD).

Exemplos sanitizados (use só o análogo)

HorSis + setHorSit

```
Se (HorSis > 870) Inicio
  HorSit = 120;   @ situação 100 @
Fim;
```

```
java.time.LocalDateTime agora = java.time.LocalDateTime.now();
int minutos = agora.getHour() * 60 + agora.getMinute();
if (minutos > 870) {
    contextoApuracao.setHorSit(100, 120);
}
```

HorSit — errado vs certo

```
// ERRADO
contexto.getSituacao(xNorAnt).setMinutos(0);
// CERTO
contexto.zeraHorasSituacao(xNorAnt);
int v = contexto.getHorSit(xExDDsr); // minutos int
contexto.setHorSit(xExDDsr, v + vNorDes);
```

Marcações FLeMar/FPxMar → List

```
for (MarcacaoRegra m : contextoApuracao.getMarcacoesRealizadas(false)) {
    LocalDate d = m.getData().toLocalDate();
    if (d.isAfter(contextoApuracao.getData())) { /* ... */ }
}
```

End → retorno (RetBHRDat)

```
// ordem Java ≠ LSP – confirmar no índice
int saldo = contextoApuracao.getSaldoBanco(codbhr, numemp, tipcol, numcad,
    new LocalDate(2014, 9, 24));
```

Múltiplos End → objeto

```
private ResultadoDiurnoNoturno calculaDiurnoNoturno(...) {
    return new ResultadoDiurnoNoturno(minutosDiurnos, minutosNoturnos);
}
```

Troca de situação

```
private int trocarSituacao(ContextoApuracao ctx, int origem, int destino, int minutosPendentes) {
    int minutosOrigem = ctx.getHorSit(origem);
    if (minutosOrigem >= minutosPendentes && minutosOrigem > 0 && minutosPendentes > 0) {
        ctx.setHorSit(destino, minutosPendentes);
        ctx.setHorSit(origem, minutosOrigem - minutosPendentes);
        return 0;
    }
    if (minutosOrigem < minutosPendentes && minutosOrigem > 0 && minutosPendentes > 0) {
        ctx.setHorSit(destino, minutosOrigem);
        ctx.setHorSit(origem, 0);
        return minutosPendentes - minutosOrigem;
    }
    return minutosPendentes;
}
```

CalculaQtdMinutos (virada de dia)

```
private int calculaQtdMinutos(LocalDate d1, int horIni, LocalDate d2, int horFim) {
    if (d1.isBefore(d2)) horFim = horFim + 1440;
    return horFim - horIni;
}
```

TipCon via ContextSession

```
int nTipCon = 0;
try {
    ResultSet rs = ContextSession.getSession().executeQuery(
        "SELECT TIPCON FROM R034FUN WHERE NUMEMP = ? AND TIPCOL = ? AND NUMCAD = ?",
        numEmp, tipCol, numCad);
    if (rs.next()) nTipCon = rs.getInt(0);
} catch (Exception ex) {
    ctx.mensagemLog("Erro TipCon: " + ex.getMessage());
}
```

Interjornada (MarcacaoAnterior)


```

if (ctx.getMarcacaoAnterior() != null) {
    int n = ctx.getMarcacaoAnterior().diferencaMinutos(datMar, horaMar);
    if (n < 0) n = -n;
    // preferir getHorasInterjornada* quando disponível no contexto
}

```

EntitySession / ICursor (USU_* sem API)

```

ICursor<IEntidadeCustom> cursor = getContainer().getEntitySession()
    .newCursor(IEntidadeCustom.class);
try {
    cursor.addFilter("USU_Campo = :v", new MappedParamProvider("v", valor));
    cursor.open();
    if (cursor.first()) { /* read */ }
} finally {
    cursor.close();
}

```

Justificar ausência de API semântica; emitir interface+ .sc só para USU_* .

DiferencaMarcacaoDiurnoNoturno (sanitizado)

```

private int[] diferencaMarcacaoDiurnoNoturno(int nMar1, LocalDate d1, int nMar2,
LocalDate d2,
                                     int iniNot, int fimNot) {
    // retorna [diurno, noturno]; usar CalculaQtdMinutos + janela 1320/300
    // implementação completa: analogia ao playbook de compilação – sem sits de
    cliente
    return new int[]{0, 0}; // preencher com a lógica do bloco LSP correspondente
}

```

Preferir getHorasSeparadas / APIs oficiais quando cobrirem o caso.

VerificaInterjornada (MarcacaoAnterior)

```

private void verificaInterjornada(ContextoApuracao ctx, LocalDate datMar, int
horaMar, int sitInt) {
    if (ctx.getMarcacaoAnterior() == null) return;
    int n = ctx.getMarcacaoAnterior().diferencaMinutos(datMar, horaMar);
    if (n < 0) n = -n;
    if (n > 0 && n < 660) {
        ctx.setHorSit(sitInt, 660 - n);
    }
    // Preferir getHorasInterjornadaRealizada/Prevista quando existirem no contexto
}

```

MarcacoesInvalidas (padrão)

```
private boolean marcacoesInvalidas(List<MarcacaoRegra> mars) {
    if (mars == null || mars.isEmpty()) return true;
    for (int i = 0; i + 1 < mars.size(); i += 2) {
        // validar pares entrada/saída – regras de negócio = validacao_manual /
        anexo
    }
    return false;
}
```

Outros pontos de customização (referência, não fluxo de menu)

Tipo	Base	APIs típicas
InicioCalculoColaborador	InicioCalculoColaborador	getMarcacoes, getTrocaHorario, incluirTrocaHorario, Periodo.getInstance
AposGravarApuracao	AposGravarApuracao	isTelaAcerto, getDiasCalculados, isUltimoColaborador
Manutenção BH	RegraManutencaoBH	getLancamentos() → ILancamentoBH
Fechamento BH	RegraFechamentoBH	getBancoHoras, getDataFinal, realizarFechamento
ContextoGeralRH	qualquer regra	getUsuarioAtivo, getUsuarioColaborador, getSaldoBancoHoras, getMesData

Checklist pré-entrega (operacional)

- ☐ execute() sem throws Exception
- ☐ import com.senior.dataset.IEntity (não g5...)
- ☐ Sem .sc em R*; .sc JSON com { ; id = filename
- ☐ MarcacaoRegra.getData().toLocalDate(); MarcacaoAnterior só diferencaMinutos
- ☐ TipCon via SQL; IDs usuário long
- ☐ Filename = classe; auxiliares no nível da classe
- ☐ ICursor close no finally; DBCenter session.close; ContextSession sem close

Itens pendentes de confirmação (⚠)

Item	Status
Package/enum completo TipoHoraExtra	⚠ Pendente — validacao_manual
@Transactional vs container	⚠ Pendente
Logico / Escolha/Caso LSP	⚠ Pendente
isNull/setNull em LocalDate de IEntity	⚠ Confirmar no projeto
Capitalização getters USU_*	⚠ Confirmar com analista

Não inventar esses itens como confirmada .

Saída para a Skill 1

```
contexto: Apuracao | FechamentoBH | outro | indefinido
item_lsp: ...
equivalente_java: ...
evidencia: confirmada | padrao_compilacao | padrao_anexo | validacao_manual
fonte: equivalencia-funcoes-regras | indice-funcoes | compilacao | anexo
limite: ...
```

Relacionados

Skill 2 (URLs/aliases) · Skill 1 · Skill 5 (`CHK-SITAPI` , `CHK-THROWS` , `CHK-SCNAT` , `CHK-TIPCON` , `CHK-MARANT` , ...)